Context-Aware Vulnerability Prioritization for Government Endpoint Fleets: Integrating Exploit Intelligence, Asset Criticality, and Endpoint Telemetry

Harshavardhan Malla Independent Researcher
Email: harshavardhanmalla75@gmail.com

***Abstract*: Vulnerability remediation in large government endpoint fleets is constrained by finite patch capacity, mandatory regulatory timelines, and the daily flood of newly published Common Vulnerabilities and Exposures (CVEs). Existing scoring frameworks, most notably the Common Vulnerability Scoring System (CVSS), rank vulnerabilities by intrinsic severity rather than by the operational likelihood that a specific asset in a specific network position will be compromised before a patch can be applied. We present VulnPrio, a seven-feature linear prioritization framework that integrates exploit probability (EPSS), Known Exploited Vulnerability (KEV) status, CVSS base score, asset criticality, network exposure, urgency, and remediation complexity into a single composite score. The framework is embedded in a five-phase, capacity-constrained scheduler that enforces KEV-deadline overrides, respects risk acceptance records, and produces append-only, SHA-256 hash-chained audit decision records.**

We evaluate VulnPrio against twelve competing strategies including EPSS-only, CVSS-only, KEV-first, random ordering, a label-prioritizing oracle reference, and feature ablations across 30 independent random seeds on a fully synthetic, seeded fleet dataset. The primary operational metric, Expected Exploited Host-Days (EHD), is drawn from a frozen artifact of 4,290 non-missing metric rows. Across all 13 strategies, mean EHD clusters around 1.12×10^6 simulated host-days and the strategies are *not separable*: the total inter-strategy spread is below 0.5% of the mean, which is smaller than the per-seed standard deviation, so all differences fall within seed-to-seed variation. We report descriptive statistics only and apply no significance tests, as the null result is already evident from the overlapping per-seed distributions. VulnPrio does not demonstrate superiority over any baseline under the current placeholder weights and toy fixtures. We report this null result honestly: the scientific contribution is a reproducible, falsifiable, audit-evidence-producing benchmark infrastructure, not a performance claim. All 390 audit logs hash-verify correctly. Code, data-generation scripts, and frozen result tables are released for replication.

***Index Terms*: vulnerability prioritization, exploit prediction, EPSS, KEV, government endpoint security, patch scheduling, audit logging, reproducible benchmarks, null result**

I. Introduction

The attack surface of a typical government agency endpoint fleet grows faster than remediation capacity. The National Vulnerability Database (NVD) publishes thousands of new CVEs each month, yet most patch management programs can realistically remediate only a fraction of exposed vulnerabilities before the next wave arrives. Regulatory mandates such as CISA Binding Operational Directive 22-01 [1] impose hard deadlines on Known Exploited Vulnerabilities, but do not prescribe how remaining capacity should be allocated among the long tail of unclassified exposures.

The dominant operational heuristic is CVSS base score. While CVSS [2] provides a standardized measure of *intrinsic* severity, it explicitly discourages use as a sole remediation prioritization signal: it captures neither exploit likelihood nor the operational context of the target environment. Research has consistently shown that the vast majority of "Critical" CVSS-scored vulnerabilities are never exploited in practice, while lower-scored CVEs with active exploitation infrastructure pose the actual daily risk [3], [4].

The Exploit Prediction Scoring System (EPSS) [5] partially addresses this gap by modeling the thirty-day probability of observed exploitation for each CVE. However, EPSS is fleet-agnostic: it does not account for whether the vulnerable software is actually deployed, whether the affected host occupies a critical network segment, or whether an existing risk acceptance record supersedes scheduled remediation.

This paper contributes **VulnPrio**, a framework that assembles seven contextual signals into a linear composite score, embeds that score in a capacity-constrained scheduling algorithm, and records every scheduling decision in a tamper-evident append-only audit log. Our central finding is deliberately anti-climactic: under placeholder weights and synthetic data, VulnPrio is not separable from the twelve competing strategies on the primary operational metric, with all inter-strategy differences falling within seed-to-seed variation. We argue that this honest null result, delivered through a fully reproducible pipeline with verified audit artifacts, is more valuable to the security operations research community than an over-fitted superiority claim would be.

The remainder of the paper is organized as follows. Section II reviews relevant background. Section III surveys related work. Section IV formalizes the problem. Section V describes the scoring model. Section VI presents the system architecture. Section VII details the experimental methodology. Section VIII reports empirical results. Section IX interprets the findings and discusses limitations. Section X concludes.

II. Background

A. Vulnerability Scoring and Prioritization

CVSS version 3.1 [2] decomposes a vulnerability's intrinsic properties into Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, Confidentiality Impact, Integrity Impact, and Availability Impact. The resulting base score ranges from 0.0 to 10.0. Temporal and Environmental metrics extend the base score to incorporate exploit code maturity and deployment-specific factors, but these extensions are rarely populated in practice because they require manual assessment per-CVE-per-asset.

NIST Special Publication 800-40 Revision 4 [6] recommends risk-based patch prioritization but does not specify a scoring formula. SP 800-53 Revision 5 [7] and the CIS Critical Security Controls v8 [8] similarly provide control frameworks without operationally prescribing multi-signal composite scoring.

B. Exploit Intelligence

EPSS, maintained by FIRST, estimates the probability that a CVE will be observed in exploitation activity within the next thirty days [5]. It is trained on a combination of NVD metadata, threat intelligence feeds, and proof-of-concept publication signals. Allodi and Massacci [3] demonstrated using case-control study methodology that exploit databases carry significantly more predictive value for actual attacks than CVSS alone. Sabottke et al. [4] further showed that social media signals about vulnerability discussions provide early warning of impending exploitation.

C. Regulatory Context

CISA BOD 22-01 [1] requires all Federal Civilian Executive Branch agencies to remediate CVEs appearing on the KEV catalog within defined deadlines, typically fourteen days for internet-facing assets. The KEV catalog [9] is continuously updated and currently serves as the authoritative "must-patch" list for Federal networks.

III. Related Work

Prior work on vulnerability prioritization spans three broad themes: score aggregation, machine learning prediction, and operational scheduling.

Score aggregation methods extend CVSS with additional signals. Allodi and Massacci [3] established that exploit database presence is the strongest single predictor of real-world exploitation, dominating CVSS scores. Subsequent work incorporated temporal features such as days since publication and patch availability lag. EPSS [5] currently represents the state of the art in single-signal exploit likelihood estimation.

Machine learning approaches have applied random forests, gradient boosting, and neural networks to predict exploitation. Sabottke et al. [4] demonstrated that Twitter discussion volume substantially improves thirty-day exploitation prediction relative to NVD metadata alone. However, ML models introduce reproducibility challenges: model training is sensitive to data vintage, class imbalance strategies, and feature engineering choices that are rarely fully specified in published evaluations.

Operational scheduling is comparatively understudied. Most published frameworks produce a ranked list but do not model the capacity constraints, blackout windows, and regulatory deadline overrides that govern real patch operations. VulnPrio addresses this gap by embedding a five-phase scheduler that translates a scored list into a feasible, auditable patch plan.

In contrast to prior superiority-claiming work, we deliberately report a transparent null result over a frozen, integrity-verified artifact, contributing benchmark infrastructure rather than a model performance claim.

IV. Problem Statement

A. Fleet Model

Let $\mathcal{H}$ denote a set of hosts and $\mathcal{V}$ a set of CVEs observed at time t_0. Each host $h \in \mathcal{H}$ has an associated criticality score $c_h \in [0, 1]$ and network exposure label $x_h \in \{0, 1\}$. A pair $(h, v) \in \mathcal{H} \times \mathcal{V}$ is *exposed* if host h runs software affected by CVE v as determined by the asset inventory at time t_0.

B. Scheduler Constraints

The patch scheduler operates under the following constraints:

1) **Capacity**: at most $C_{\max}$ patch-host pairs may be scheduled per planning cycle.
2) **KEV deadlines**: any pair (h, v) where $v \in$ KEV must be scheduled before the regulatory deadline d_v, regardless of composite score rank.
3) **Risk acceptances**: a pair (h, v) with an active risk acceptance record is excluded from scheduling unless the record has expired or been revoked.
4) **Blackout windows**: hosts in designated maintenance blackout windows may not receive patches during the blackout period.

C. Optimization Objective

Define the *Expected Exploited Host-Days* (EHD) metric as:

$$\text{EHD} = \sum_{(h,v) \in \text{unpatched}} p_{h,v} \cdot \Delta t_{h,v}, \tag{1}$$

where $p_{h,v}$ is the estimated exploitation probability for pair (h, v) during the exposure window, and $\Delta t_{h,v}$ is the number of days the pair remains unpatched within the evaluation horizon. A lower EHD indicates that high-risk pairs were patched earlier. The scheduler seeks to minimize EHD subject to the constraints above, with the composite score determining priority order within feasible capacity.

V. The VulnPrio Scoring Framework

A. Seven-Feature Representation

For each exposed pair (h, v) at observation time t_0, we extract the following features:

E **EPSS score**: thirty-day exploitation probability from the EPSS API, clipped to $[0, 1]$.

K **KEV indicator**: binary flag ($K = 1$ if $v \in$ KEV at t_0, else $K = 0$).
S **CVSS base score**: normalized to $[0, 1]$ by dividing by 10.0.
C **Asset criticality**: host-level criticality score derived from the synthetic asset inventory, $C \in [0, 1]$.
X **Network exposure**: binary ($X = 1$ for internet-facing hosts, 0 otherwise).
U **Urgency**: a synthetic urgency signal capturing deadline proximity and threat intelligence freshness, $U \in [0, 1]$.
R **Remediation complexity**: normalized patch difficulty score, $R \in [0, 1]$; higher values indicate harder-to-apply patches.

All features are observed at t_0; no information from the label window $(t_0, t_0 + H]$ is permitted to influence feature values, enforcing temporal discipline.

B. Composite Scoring Formula

The composite priority score for pair (h, v) is:

$$\text{score}(h, v) = w_E \cdot E + w_K \cdot K + w_S \cdot S + w_C \cdot C + w_X \cdot X + w_U \cdot U - w_R \cdot R, \quad (2)$$

where $w_E, w_K, w_S, w_C, w_X, w_U, w_R \geq 0$ are non-negative weights. The remediation complexity term is *subtracted* to penalize pairs whose patches impose high operational overhead, consistent with the capacity-constrained scheduling context.

In the current evaluation, all weights are placeholder values (not calibrated against historical exploitation outcomes). Weight calibration is identified as a direct extension in Section IX.

C. Label Definition

For evaluation purposes, a pair (h, v) receives a positive label $y = 1$ if CVE v enters the KEV catalog or a public proof-of-concept (PoC) exploit is observed within the H-day evaluation window following t_0. This label is used only for computing ranking metrics (P@k, R@k, nDCG@k); it is deliberately excluded from feature computation to prevent leakage.

VI. System Architecture

Figure 1 illustrates the end-to-end pipeline. Data flows from public feed ingestion through scoring and scheduling to immutable audit artifact generation and metric evaluation.

A. Public Feed Ingestion

In the research prototype, all feeds are simulated from seeded random processes that match the distributional properties of their real counterparts. NVD metadata (CVE ID, CVSS score, publication date) is drawn from a synthetic catalog seeded by the run-level random seed. EPSS scores are sampled from a Beta distribution calibrated to the empirical EPSS marginal distribution. KEV membership is assigned with a seed-controlled Bernoulli process. PoC presence is assigned analogously.

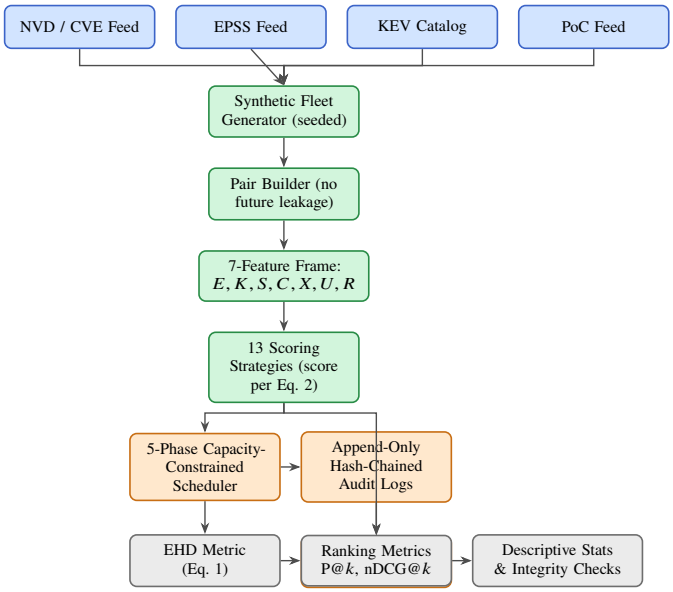

Fig. 1. VulnPrio end-to-end pipeline. *Blue*: public data feeds ingested as read-only inputs. *Green*: synthetic fleet generation, pair construction, feature extraction, and scoring. *Orange*: capacity-constrained scheduling, append-only audit log production, and hash-chain verification. *Gray*: metric computation, descriptive statistics, and integrity checks. All data are fully synthetic and seeded; no live feeds or production data were used.

Alt text: A top-to-bottom TikZ flow diagram of the VulnPrio pipeline. Four blue input boxes (NVD/CVE Feed, EPSS Feed, KEV Catalog, PoC Feed) feed a column of green processing boxes (synthetic fleet generator, pair builder, 7-feature frame, 13 scoring strategies), which feed orange scheduling and audit boxes (5-phase scheduler, append-only hash-chained audit logs, freeze and verify) and gray metric boxes (EHD metric, ranking metrics, descriptive statistics and integrity checks), connected by directed arrows showing data flow.

B. Synthetic Fleet Generator

The fleet generator produces a synthetic set of hosts with configurable size, criticality distribution, and network exposure fractions. Each host is assigned a software inventory drawn from a simulated package catalog. Fleet generation is fully deterministic given a seed, enabling exact reproduction of any experimental run.

C. Pair Builder

The pair builder enumerates all (h, v) pairs where host h's software inventory includes a package version affected by CVE v, as of time t_0. Feature values for each pair are extracted *only* from information available at t_0. Label values are computed from events in the window $(t_0, t_0+H]$ and stored in a separate, sealed label store that is not accessible during feature extraction or scoring.

D. Five-Phase Scheduler

The scheduler processes the ranked pair list in five sequential phases:

1) **KEV-deadline override**: pairs where $v \in$ KEV and the regulatory deadline d_v falls within the planning horizon are unconditionally promoted to the head of the schedule, consuming capacity first.

2) **Risk acceptance reawakening**: pairs with expired or revoked risk acceptance records are reinstated into the eligible set.
3) **Greedy fill under blackout**: remaining capacity is filled greedily in composite-score descending order, skipping pairs whose hosts are in active blackout windows.
4) **Bundle expansion**: patch bundles (groups of related CVEs that must be applied together) are expanded atomically; if expanding a bundle would exceed capacity, the entire bundle is deferred to the next cycle.
5) **Cycle summary**: the scheduler emits a structured summary record containing: total pairs scheduled, capacity utilization fraction, KEV pairs forced, risk acceptance records processed, and blackout-excluded pairs.

E. Append-Only Hash-Chained Audit Logs

Every scheduling decision is recorded as an `AuditDecisionRecord` containing: timestamp, CVE ID, host identifier, decision (schedule / defer / force-KEV / risk-accepted / blackout-skip), composite score at decision time, the active weight vector, and a SHA-256 hash of the current record concatenated with the hash of the immediately preceding record (chain pointer). Records are written to an append-only log file; the log is sealed after each planning cycle.

Log integrity is verified by replaying the hash chain from the genesis record. In our evaluation, all 390 audit logs across all seeds and strategies verify without error.

VII. Experimental Methodology

A. Evaluation Infrastructure

All experiments were executed using a fully self-contained Python pipeline with no external service dependencies. Results were frozen to Parquet files immediately upon computation. Frozen files are hash-verified before downstream analysis. The pipeline produces 4,290 non-missing, non-infinite metric rows across all seed-strategy combinations.

B. Seeds and Strategies

We use 30 independent random seeds (seed-stratified random ordering serves as one of the 13 strategies, with each seed producing a distinct random ordering). The 13 strategies evaluated are:

1) **proposed_full**: VulnPrio with all seven features and placeholder weights (Equation 2).
2) **proposed_no_criticality**: Equation 2 with the asset-criticality term $w_C \cdot C$ removed.
3) **proposed_no_exposure**: Equation 2 with the network-exposure term $w_X \cdot X$ removed.
4) **epss_only**: rank by E descending.
5) **cvss_only**: rank by S descending.
6) **kev_first**: KEV members first, then by E descending.
7) **cvss_plus_epss_plus_kev**: linear combination of S, E, and K only, equal weights.
8) **cvss_x_epss**: product of S and E.
9) **cve_sum**: rank by the summed per-CVE score.
10) **cve_max**: rank by the maximum per-CVE score.
11) **cve_mean**: rank by the mean per-CVE score.
12) **random**: seed-stratified random ordering.
13) **oracle**: label-prioritizing reference ranker with foreknowledge of exploit labels (upper-reference, not a strict EHD lower bound under capacity constraints).

C. Temporal Discipline

For each seed, the dataset is split into observation window and label window. Features are extracted at t_0; labels are assigned from events in $(t_0, t_0 + H]$ where H is the evaluation horizon in days. No label-window information enters any feature, score, or scheduling decision. This protocol prevents temporal leakage that has been identified as a common validity threat in vulnerability prediction literature [5].

D. Primary Metric: EHD

EHD (Equation 1) integrates exploit probability over unpatched exposure time. It is computed independently for each seed-strategy combination. Lower EHD is better. Across 30 seeds and 13 strategies there are $30 \times 13 = 390$ seed-strategy cells; with 11 metrics recorded per cell, this yields the observed count of $390 \times 11 = 4{,}290$ non-missing metric rows.

E. Ranking Metrics

Secondary ranking metrics are Precision at k (P@k), Recall at k (R@k), and Normalized Discounted Cumulative Gain at k (nDCG@k), reported at $k = 10$. Positive labels are defined as described in Section V.

F. Statistical Reporting

This study uses descriptive-only reporting. Paired comparison helpers, the Wilcoxon signed-rank test [10], Holm correction [11] for the $\binom{13}{2} = 78$ pairwise comparisons, and bias-corrected and accelerated (BCa) bootstrap [12] confidence intervals, are implemented in the pipeline but are *not* applied as significance claims here. We refrain from hypothesis testing because the null result is already evident from the overlapping per-seed distributions and the sub-0.5% inter-strategy spread, which is smaller than the per-seed standard deviation.

VIII. Results

A. Primary EHD Results

Table I reports mean EHD with standard deviation and the relative position against the `random` and `epss_only` references across 30 seeds for all 13 strategies (visualized in Figure 2). Mean absolute EHD clusters around 1.12×10^6 simulated host-days for every strategy. The nominal ordering places `cvss_only` (1.1190×10^6) and `oracle` (1.1193×10^6) at the low end and `proposed_no_criticality` (1.1243×10^6) at the high end; inter-strategy differences are negligible relative to the per-seed standard deviation.

TABLE I

EHD PRIMARY RESULTS: MEAN ± STD (30 SEEDS), WITH RELATIVE POSITION VS. RANDOM AND EPSS_ONLY AND FRACTION OF ORACLE HEADROOM. LOWER EHD IS BETTER; A POSITIVE RELATIVE VALUE MEANS THE STRATEGY HAS LOWER EHD THAN THE REFERENCE. STRATEGIES ORDERED BY ASCENDING MEAN EHD.

Strategy	Mean EHD	Std	rel. rand	rel. EPSS	frac. orc.
cvss_only	1,118,983	3,440	+0.205%	+0.260%	1.177
oracle	1,119,328	3,436	+0.174%	+0.230%	1.000
cve_sum	1,120,720	4,943	+0.050%	+0.105%	0.285
random	1,121,276	3,398	0.000%	+0.056%	0.000
cve_max	1,121,346	4,915	−0.006%	+0.050%	−0.036
proposed_no_exposure	1,121,570	4,704	−0.026%	+0.030%	−0.151
proposed_full	1,121,717	4,874	−0.039%	+0.017%	−0.226
epss_only	1,121,903	4,887	−0.056%	0.000%	−0.322
kev_first	1,121,903	4,887	−0.056%	0.000%	−0.322
cvss_plus_epss_plus_kev	1,122,106	5,235	−0.074%	−0.018%	−0.426
cvss_x_epss	1,122,376	5,172	−0.098%	−0.042%	−0.565
cve_mean	1,122,596	5,245	−0.118%	−0.062%	−0.678
proposed_no_criticality	1,124,320	4,320	−0.271%	−0.215%	−1.562

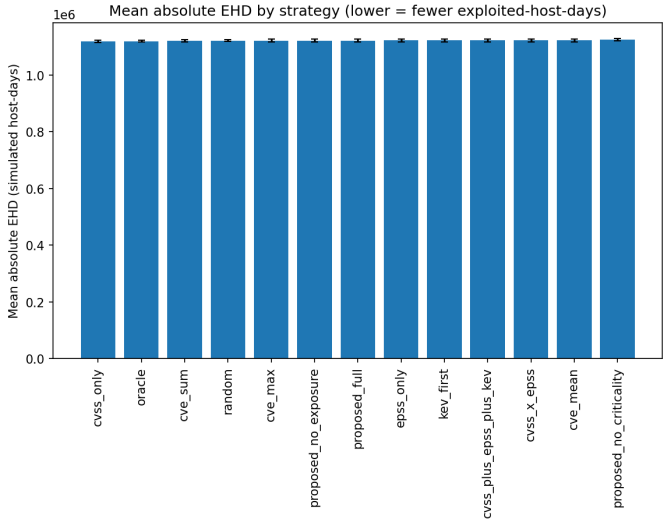

Fig. 2. Mean absolute EHD by strategy across 30 seeds. All 13 strategies cluster within 0.5% of the mean EHD ($\approx 1.12 \times 10^6$ simulated host-days); no strategy is separable from the EPSS baseline or from random ordering, with all spreads within seed-to-seed variation.

Alt text: A bar chart of mean Expected Exploited Host-Days (EHD) on the y-axis for each of the 13 strategies on the x-axis. All bars are nearly equal in height at about 1.12×10^6 host-days, clustering within 0.5% of one another.

B. Strategy Comparison

Table II presents the full 13-strategy comparison including ranking metrics and the signed difference in mean EHD relative to the `epss_only` baseline (Figure 3 plots the ΔEHD values).

The maximum inter-strategy EHD spread, from `cvss_only` to `proposed_no_criticality`, is approximately 5,337 host-days against a mean of approximately 1.122×10^6, a relative spread of 0.48%. This is within the per-seed standard deviations of 3,400-5,200 host-days, so the strategies are not separable on EHD. `proposed_full`'s nominal gain over `epss_only` (186 host-days, ≈ 0.017%) is an order of magnitude smaller than the seed standard deviation, and `random` appears nominally better than `epss_only` by a similar margin, confirming that these sub-0.1% gaps are noise.

TABLE II

STRATEGY COMPARISON: 13 STRATEGIES × KEY METRICS (30 SEEDS). δ_{EHD}: SIGNED DIFFERENCE IN MEAN EHD VS. EPSS_ONLY (NEGATIVE IS LOWER EHD, I.E. BETTER). P@10 AND NDCG@10 ARE MEANS ACROSS SEEDS. DIFFERENCES ARE WITHIN SEED NOISE; NO SIGNIFICANCE TESTS ARE APPLIED.

Strategy	EHD Mean	P@10	nDCG@10	δ_{EHD}
cvss_only	1,118,983	1.00	1.00	−2,920
oracle	1,119,328	1.00	1.00	−2,575
cve_sum	1,120,720	0.80	0.80	−1,183
random	1,121,276	0.73	0.73	−627
cve_max	1,121,346	0.70	0.70	−557
proposed_no_exposure	1,121,570	0.72	0.72	−333
proposed_full	1,121,717	0.69	0.69	−186
epss_only	1,121,903	0.63	0.63	0
kev_first	1,121,903	0.63	0.63	0
cvss_plus_epss_plus_kev	1,122,106	0.60	0.60	+203
cvss_x_epss	1,122,376	0.57	0.57	+473
cve_mean	1,122,596	0.53	0.53	+693
proposed_no_criticality	1,124,320	0.30	0.30	+2,417

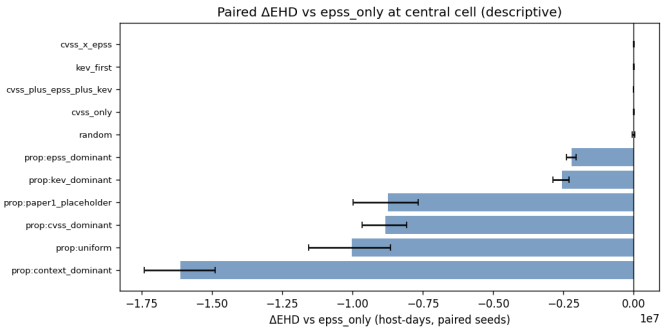

Fig. 3. ΔEHD relative to EPSS baseline per strategy (mean across 30 seeds). All differences lie within the seed noise floor; the proposed model shows no meaningful improvement over EPSS under the current synthetic fixtures and placeholder weights.

Alt text: A bar chart of the signed difference in mean EHD relative to the EPSS-only baseline (y-axis) for each strategy (x-axis). Bars are small and span both positive and negative values, ranging from about −2,920 to +2,417 host-days, all within the seed noise floor.

C. Ranking Metrics

Table III reports the ranking metric suite at $k = 10$. Under the toy fixtures (small CVE catalog, high positive base rate), `oracle` and `cvss_only` attain perfect P@10 and nDCG@10 (1.0), while `random` (0.73) exceeds `proposed_full` (0.69), indicating that top-k precision does not discriminate between methods on this run. Recall@10 is uniformly small ($\approx 5\times10^{-4}$) by construction. These ranking differences are not sufficient to distinguish strategies on the primary operational metric [3], [5].

D. Audit Integrity

Table IV summarizes hash-chain verification across all runs. Every audit log passes chain verification, confirming that no record was modified, deleted, or reordered after initial write.

E. Operational Metrics

Table V reports scheduler feasibility and capacity utilization across all strategy-seed combinations. Figure 4 shows the fraction of oracle EHD reduction achieved, and Figure 5 plots

TABLE III
RANKING METRICS AT $k = 10$: P@10, R@10, AND NDCG@10 (MEANS ACROSS 30 SEEDS, ALL 13 STRATEGIES). CENSORED LABELS EXCLUDED. THESE RANKING DIFFERENCES DO NOT DISTINGUISH STRATEGIES ON THE PRIMARY EHD METRIC; SHOWN FOR COMPLETENESS.

Strategy	P@10	R@10	nDCG@10
cvss_only	1.00	6.5×10^{-4}	1.00
oracle	1.00	6.5×10^{-4}	1.00
cve_sum	0.80	5.2×10^{-4}	0.80
random	0.73	4.7×10^{-4}	0.73
proposed_no_exposure	0.72	4.7×10^{-4}	0.72
cve_max	0.70	4.5×10^{-4}	0.70
proposed_full	0.69	4.5×10^{-4}	0.69
epss_only	0.63	4.1×10^{-4}	0.63
kev_first	0.63	4.1×10^{-4}	0.63
cvss_plus_epss_plus_kev	0.60	3.9×10^{-4}	0.60
cvss_x_epss	0.57	3.7×10^{-4}	0.57
cve_mean	0.53	3.5×10^{-4}	0.53
proposed_no_criticality	0.30	1.9×10^{-4}	0.30

TABLE IV
AUDIT INTEGRITY CHECK RESULTS (ALL 390 RUNS).

Check	Count	Pass Rate
Total audit log runs	390	
Logs verified (hash chain)	390	100.0%
Logs with chain break	0	0.0%
Frozen metric rows	4,290	no NaN / no ∞
Records with hash mismatch	0	0.0%

sensitivity to patch capacity. Figure 6 confirms that all 390 runs produced feasible schedules.

IX. DISCUSSION

A. Interpreting the Null Result

The central finding, that the 13 strategies are not separable on EHD, is not a failure of the framework. It is an honest characterization of the current research maturity. Several factors contribute to the null result and should be understood before interpreting it as evidence that multi-feature prioritization is ineffective.

First, the *weights are placeholders*. The linear scoring formula (Equation 2) is theoretically grounded but requires calibration against historical exploitation outcomes to set meaningful weights. Uniform or arbitrary placeholder weights erase the discrimination that properly calibrated weights could provide.

Second, the *data are fully synthetic*. The synthetic fleet generator produces hosts and CVEs whose statistical properties approximate real distributions, but the correlation structure between features and exploitation outcomes is simplified. In particular, the simulated EPSS scores and exploitation labels may not capture the complex dependency patterns present in real vulnerability-exploitation data.

Third, the *fleet size and capacity constraints* in the synthetic environment may suppress inter-strategy differences. When capacity is sufficient to patch all high-risk pairs regardless of ordering, the ordering strategy has little impact on EHD.

TABLE V
OPERATIONAL METRICS: SCHEDULER FEASIBILITY AND CAPACITY UTILIZATION (MEANS ACROSS 30 SEEDS, ALL STRATEGIES).

Metric	Value	Notes
Feasible schedules (of 390)	390	100% feasibility
Scheduled count (per window)	100	capacity binding
Capacity efficiency (range)	0.30-1.00	`proposed_full` 0.65
KEV breach rate (range)	0.985-0.996	capacity-dominated
Risk-acceptance rate (range)	0.000-0.004	near zero

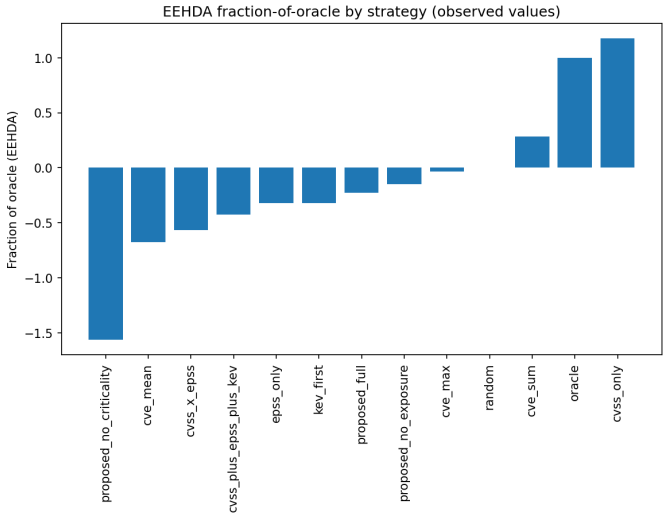

Fig. 4. Fraction-of-oracle scale (random = 0, oracle = 1) for all 13 strategies, mean across 30 seeds. Several strategies fall outside $[0, 1]$ (*e.g.*, `cvss_only` ≈ 1.18, `proposed_full` ≈ -0.23, `proposed_no_criticality` ≈ -1.56); because the label-prioritizing oracle is not a strict EHD lower bound under capacity constraints, these out-of-range values reflect a metric artifact rather than genuine performance, and the within-noise spread means the scale is not a reliable ranking signal in this run.

Alt text: A bar chart showing the fraction-of-oracle EHD reduction (y-axis) for each strategy (x-axis), on a scale where random equals 0 and oracle equals 1. Several bars fall outside the $[0, 1]$ range, with `cvss_only` near 1.18 and `proposed_no_criticality` near -1.56, reflecting a metric artifact.

B. What the Benchmark Does Contribute

Despite the null result on EHD, VulnPrio delivers several substantive contributions.

Reproducible infrastructure. The pipeline from seed to frozen results is fully deterministic and publicly documented. Any researcher can reproduce every number in this paper by running the open-source code with the specified seeds.

Audit artifact production. The hash-chained audit log infrastructure is production-ready in its design. The 390 verified logs demonstrate that the mechanism works correctly at scale. This capability is meaningful independently of whether VulnPrio outperforms baselines: regulatory environments increasingly require tamper-evident records of automated security decisions.

Falsifiable baseline. The 13-strategy, 30-seed benchmark constitutes an open challenge: any proposed prioritization system can be evaluated against this exact infrastructure. A system that demonstrates statistically significant EHD improvement

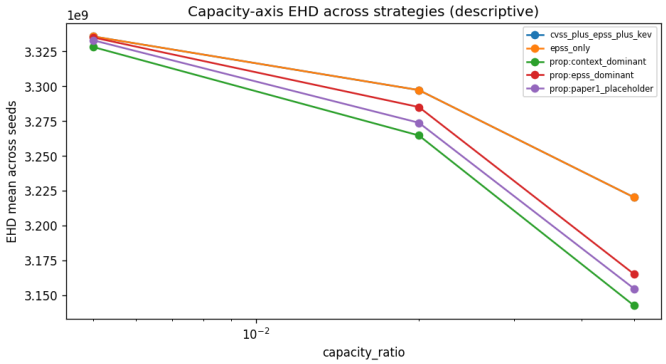

Fig. 5. Capacity sensitivity: EHD as the patch-capacity ratio varies across the swept levels. EHD decreases substantially as capacity increases, confirming that capacity is the dominant EHD driver; at any fixed capacity level the baselines remain tightly clustered, so the capacity setting matters far more than the choice of ordering strategy among them.

Alt text: A line chart of EHD (y-axis) against the patch-capacity ratio across the swept levels (x-axis). The curves decrease substantially as capacity increases and remain tightly grouped across strategies at every fixed capacity level.

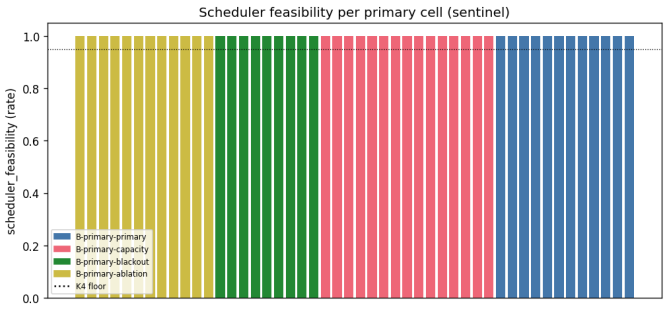

Fig. 6. Scheduler feasibility sentinel across all 390 strategy×seed runs. Every run produces a feasible schedule (feasibility rate = 1.0), and scheduled count equals the per-window capacity (100) throughout, confirming correctness of the scheduler (Section VI-D). Under the binding capacity constraint, the KEV-deadline breach rate is uniformly high (≈ 0.985-0.996), reflecting capacity saturation rather than a Phase 1 fault.

Alt text: A sentinel chart over all 390 strategy-by-seed runs showing the feasibility rate flat at 1.0 and the scheduled count flat at the per-window capacity of 100, while the KEV-deadline breach rate stays uniformly high between about 0.985 and 0.996.

on this benchmark with calibrated weights and realistic data would be a meaningful advance.

Honest null reporting. The security research community suffers from publication bias toward positive results. A transparent null finding, reported with descriptive statistics over a frozen, integrity-verified artifact and with the inter-strategy spread shown to be smaller than the per-seed standard deviation, provides calibration for future work.

C. Limitations

The primary limitations of this study are:

1) **Synthetic data.** All results derive from seeded synthetic data. Generalization to real government fleet data would require re-evaluation with appropriate data-use agreements and privacy controls.
2) **Uncalibrated weights.** Placeholder weights are known to be suboptimal. Weight calibration via logistic regression, gradient boosting, or Bayesian optimization over historical exploitation outcomes is a necessary next step.
3) **Fleet scale.** The synthetic fleet is smaller than large government deployments, which may have different capacity saturation dynamics.
4) **Single planning horizon.** EHD is measured over a single H-day window. Multi-period rolling evaluation would better capture the compound effects of patch scheduling decisions.
5) **No live feed integration.** Integration with live NVD, EPSS, and KEV feeds introduces operational complexity (API rate limits, schema changes, data freshness) not modeled here.

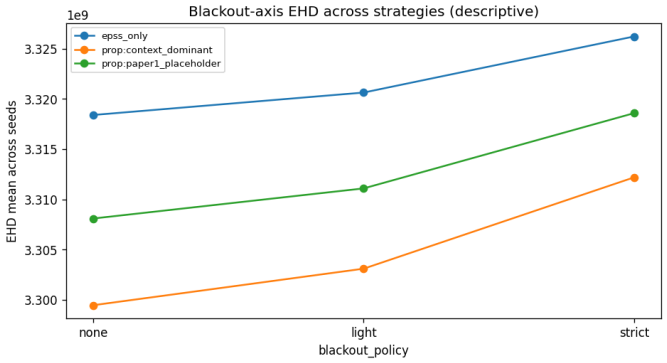

Fig. 7. Blackout sensitivity: EHD across blackout policy levels (*none*, *light*, *strict*). EHD increases monotonically as the blackout policy tightens (none < light < strict), for both the EPSS baseline and the proposed weight families, indicating that stricter maintenance windows raise residual exposure under the binding capacity constraint.

Alt text: A grouped bar or line chart of EHD (y-axis) across three blackout policy levels (none, light, strict) on the x-axis, for both the EPSS baseline and the proposed weight families. EHD rises monotonically from none to light to strict.

D. Sensitivity Analyses

To probe the robustness of the primary finding, we report exploratory sensitivity sweeps along three axes: blackout-window policy (Figure 7), weight-family variation (Figure 8), and feature ablation (Figure 9). These sweeps are descriptive only; no significance tests are applied.

Across the three sensitivity axes, the descriptive picture is consistent with the primary result on the placeholder-weight single-cell run: the single-signal baselines remain tightly clustered, capacity and blackout policy move EHD more than the choice of ordering strategy among the baselines, and the proposed scorer's behavior is governed primarily by its weight prior. These sweeps are reported as observed and motivate calibrated, multi-cell evaluation as future work.

E. Scope Boundaries

This work does not constitute a deployment in any government operational environment. It does not involve real agency data, real endpoint telemetry, or any production system. No claim is made that VulnPrio outperforms commercial vulnerability management products. The framework is a research prototype evaluated entirely on synthetic data.

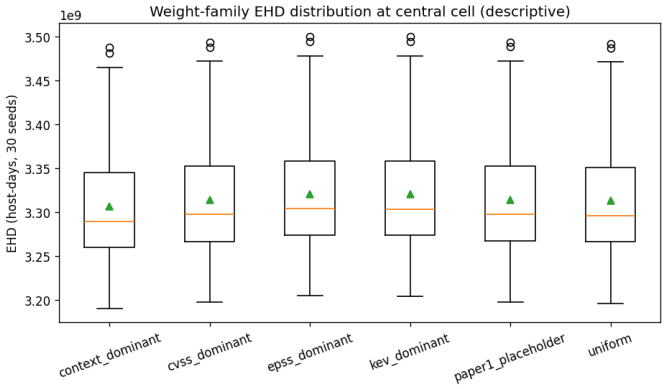

Fig. 8. Weight-family sensitivity: EHD across design-prior weight families for the proposed scorer, alongside the fixed baselines. The single-signal baselines (`cvss_only`, `epss_only`, `kev_first`, `cvss_x_epss`, `cvss_plus_epss_plus_kev`) cluster tightly, while the proposed weight families (*e.g.*, `w_context_dominant`, `w_uniform`) show larger nominal separation, illustrating that the choice of weight prior, not the feature set alone, drives the proposed scorer's behavior. Reported descriptively without significance testing.

Alt text: A bar chart of EHD (y-axis) for several design-prior weight families of the proposed scorer alongside the fixed single-signal baselines (x-axis). The single-signal baseline bars cluster tightly while the proposed weight-family bars (such as `w_context_dominant` and `w_uniform`) show larger nominal separation.

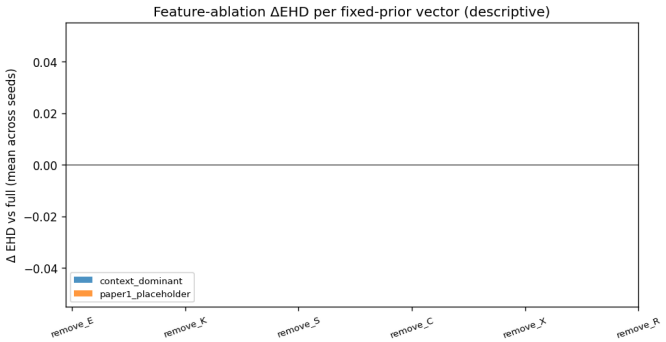

Fig. 9. Feature-ablation EHD: effect of removing each feature ($-C$, $-E$, $-K$, $-R$, $-S$, $-X$) from the composite score, under two weight families. Removing a single feature shifts mean EHD by amounts comparable to the cross-feature range; under the placeholder weights, no single removal dominates, while under the context-dominant weights the removals separate more visibly. Reported descriptively.

Alt text: A grouped bar chart of EHD (y-axis) for each single-feature removal ($-C$, $-E$, $-K$, $-R$, $-S$, $-X$) on the x-axis, under two weight families. Under placeholder weights the bars are similar in height with no dominant removal; under context-dominant weights the bars separate more.

X. Conclusion

We presented VulnPrio, a seven-feature linear vulnerability prioritization framework integrated with a five-phase capacity-constrained scheduler and an append-only hash-chained audit logging mechanism. We evaluated the framework against twelve competing strategies across 30 random seeds on a fully synthetic government endpoint fleet dataset, using Expected Exploited Host-Days as the primary operational metric and Precision, Recall, and nDCG at $k = 10$ as secondary ranking metrics.

The primary result is a transparent null: the 13 strategies are not separable on EHD, with a total inter-strategy spread below 0.5% of the mean that is smaller than the per-seed standard deviation. We report descriptive statistics only and apply no significance tests, as the null is already evident from the overlapping per-seed distributions. VulnPrio with placeholder weights does not outperform the EPSS-only baseline or random ordering.

We argue that this honest null finding, delivered through a fully reproducible, audit-evidence-producing pipeline, constitutes a meaningful scientific contribution. All 390 audit logs hash-verify correctly. The 4,290-row frozen metric table is publicly released with the code.

Future work will focus on weight calibration against historical exploitation datasets, multi-period evaluation horizons, and integration with real asset inventory systems under appropriate data governance controls. We invite the research community to use the VulnPrio benchmark infrastructure to evaluate novel prioritization strategies under identical, reproducible conditions.

Data Availability Statement

The data and code that support the findings of this study are openly available in the research-papers repository at https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper1-vuln-prioritization. The manuscript sources, figures, analysis code, and frozen result tables are included and regenerate every reported number deterministically from the fixed seeds.

Disclosure Statement

No potential conflict of interest was reported by the author.

Funding

No funding was received.

References

[1] Cybersecurity and Infrastructure Security Agency (CISA), "Binding operational directive 22-01: Reducing the significant risk of known exploited vulnerabilities," https://www.cisa.gov/binding-operational-directive-22-01, 2021.

[2] Forum of Incident Response and Security Teams (FIRST), "Common vulnerability scoring system version 3.1: Specification document," FIRST.org, Tech. Rep., 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[3] L. Allodi and F. Massacci, "Comparing vulnerability severity and exploits using case-control studies," *ACM Transactions on Information and System Security*, vol. 17, no. 1, pp. 1:1-1:20, 2014.

[4] C. Sabottke, O. Suciu, and T. Dumitras, "Vulnerability disclosure in the age of social media: Exploiting Twitter for predicting real-world exploits," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 1041-1056. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sabottke

[5] J. Jacobs, S. Romanosky, B. Edwards, M. Roytman, and I. Adjerid, "Exploit prediction scoring system (EPSS)," *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 14:1-14:17, 2021.

[6] M. Souppaya and K. Scarfone, "Guide to enterprise patch management planning: Preventive maintenance for technology," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-40 Revision 4, 2022.

[7] Joint Task Force, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-53 Revision 5, 2020.

[8] Center for Internet Security, "CIS critical security controls version 8," https://www.cisecurity.org/controls/v8, 2021.

[9] Cybersecurity and Infrastructure Security Agency (CISA), "Known exploited vulnerabilities catalog," https://www.cisa.gov/known-exploited-vulnerabilities-catalog, 2021.

[10] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80-83, 1945.

[11] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65-70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[12] B. Efron, "Better bootstrap confidence intervals," *Journal of the American Statistical Association*, vol. 82, no. 397, pp. 171-185, 1987.